Assignment Self-Consistency (Chain-of-Thought)

Read this paper first

Self-Consistency Improves Chain of Thought Reasoning in Language Models
Wang et al., ICLR 2023

- Summary: arxiv.org/abs/2203.11171
- Full PDF: arxiv.org/pdf/2203.11171.pdf

What is self-consistency? (in plain English)

Chain-of-thought (CoT) asks the model to **show its working** before giving an answer.

Problem: one attempt can still be wrong.

Self-consistency fix:

1. Ask the **same question** several times (with a bit of randomness).
2. Each time the model may reason differently.
3. Pull out the **final number** from each answer.
4. Pick the number that appears **most often** (majority vote).

That's it. No extra AI judge — just count which answer shows up the most.

What you need to build

Copy the **same layout** as the solution notebook. Use plain Python and the provider setup from Notebook 1. (No LangChain or other tool, try to use plain python with simple OpenAI SDK)

Step 0 — Setup

Write these helpers once at the top:

Function	What it does
`llm_reply(prompt, temperature=...)`	Calls the model (same registry as Notebook 1)
`cot_prompt(question)`	Wraps the question in a "think step by step" prompt. Ask the model to end with `Final answer: <number>` (see paper page 3, footnote)
`extract_final_number(text)`	Pulls the final number out of the model's text

Step 1 — Show why one answer isn't enough (offline, no API key)

Paper: page 2, Figure 1

You don't need a live model for this part.

1. Pick a maths word problem (the juggler / golf-ball question is fine — correct answer is **4**).
2. Write **5 fake model answers** by hand — some should end in 4, at least one should end in a wrong number (like 8).
3. Run `extract_final_number` on each string.
4. Count which number appears most — print the winner.

This shows how voting fixes a single bad path.

Step 2 — Wrap the vote in functions (still offline)

Paper: page 3 (majority vote + Table 1)

Build:

- `majority_vote(answers)` returns the winning number and how many votes each number got
- `VoteResult` — a small dataclass holding: winner, tally, samples, confidence
- `self_consistent_answer(reasoning_samples)` runs the vote on a list of text strings

Run it on your 5 hand-written strings from Step 1.
Use a simple count vote (most votes wins). You don't need the weighted version from the paper.

Step 3 — Ask the real model several times

Paper: page 2 (sample many paths), page 4 (`temperature = 0.7`), page 5 (paper uses 40 samples — **you use 5** to save API calls)

Build:

- `sample_cot_paths(question, n_samples=5, temperature=0.7)` — same prompt, 5 separate API calls
- `run_self_consistency(question, ...)` — samples paths, then votes on them

Add a note in markdown: the paper uses 40 samples; we use 5 because each sample costs an API call.

Step 4 — Compare: one call vs five calls

Paper: page 5 (greedy = one path at temperature 0)

On the **juggler question**, run both:

1. **One call** at `temperature=0` — print the full reply and the extracted number.
2. **Self-consistency** — 5 calls at `temperature=0.7` — print each reply, the vote tally, the winner, and confidence.

If you don't have an API key, print a friendly message and skip (Steps 1–2 should still work).

Step 5 — Try a second question

Paper: page 6 (more examples)

Run `run_self_consistency` on one more word problem — e.g. Betty's wallet question from the solution notebook.

Print the tally, winner, and confidence.

Step 6 — Short write-up (markdown only, no new code)

In a markdown cell, briefly answer:

- Single CoT = **1** API call. Self-consistency with 5 samples = **5** calls.
- When is the extra cost worth it?

Functions checklist

Your notebook should define all of these (same names as the solution):

- ☐ `cot_prompt`
- ☐ `extract_final_number`
- ☐ `llm_reply`
- ☐ `majority_vote`
- ☐ `self_consistent_answer`
- ☐ `sample_cot_paths`
- ☐ `run_self_consistency`

Important: don't ask another LLM to pick the best answer. Count votes in Python, like the paper.
